7

Part 1: Concepts to Understand

Here, I will explain the story and logic of Deadlocks so you can understand **how** and **why** they happen.

1. What is a Deadlock?

Imagine two cars meeting on a narrow bridge that allows only one car at a time. Car A cannot move forward because Car B is there. Car B cannot move forward because Car A is there. Neither car can move backward. They are stuck forever.

In a computer, this happens with **processes** (programs) and **resources** (like printers, CD drives, or memory).

- **Process A** holds a resource (Resource 1) and is waiting for Resource 2.
- **Process B** holds Resource 2 and is waiting for Resource 1.
- Because Process A will not let go of its resource until it finishes, and Process B is the same, they wait forever. This situation is called a **Deadlock**.

Important Process Flow:
A process uses a resource in this exact order (Slide 6):

1. **Request:** Ask for the resource. If it is busy, the process must wait.
2. **Use:** The process uses the resource.
3. **Release:** The process finishes and gives the resource back.

2. The Four Conditions (The "Ingredients" for Deadlock)

For a deadlock to happen, **four specific things** must be true at the exact same time (Slide 9). If you remove just one of these, a deadlock cannot happen.

1. **Mutual Exclusion:** This means "Only one person at a time." Some resources (like a printer) cannot be shared. If one process uses it, others must wait.
2. **Hold and Wait:** A process is holding at least one resource and is waiting to get more. (It does not give up what it has while it waits).
3. **No Preemption:** "Preemption" means taking something away by force. "No Preemption" means resources cannot be taken away from a process. The process must give them up voluntarily.
4. **Circular Wait:** This is a circle of waiting. Process P0 waits for P1, P1 waits for P2, and P2 waits for P0.

3. Resource Allocation Graph (RAG)

Engineers use pictures called **graphs** to see if a deadlock exists (Slides 10-14).

- **Circles:** Represent Processes (P1, P2...).
- **Squares (Rectangles):** Represent Resources (R1, R2...). Dots inside the square show how many items (instances) of that resource exist (e.g., 2 dots = 2 printers).
- **Arrows (Edges):**

Request Edge: An arrow pointing from a Process to a Resource (Pi→RjP_i \rightarrow R_jPi→Rj

  -). This means the process *wants* the resource.

Assignment Edge: An arrow pointing from a Resource to a Process (Rj→PiR_j \rightarrow P_iRj→Pi

  -). This means the process *has* the resource.
-

The Rule of Cycles (Circles):

- If you follow the arrows and they form a closed loop (a circle), there *might* be a deadlock.
- If the resource type has only **one instance** (one dot) and there is a cycle, **Deadlock is certain.**
- If the resource type has **multiple instances** (many dots) and there is a cycle, **Deadlock is possible**, but not 100% sure.

4. How to Handle Deadlocks

Operating systems have three main ways to handle this problem (Slide 22):

1. **Deadlock Prevention or Avoidance:** Ensure the system never enters a deadlock.
2. **Deadlock Detection and Recovery:** Let deadlock happen, then find it and fix it.
3. **Ignore the problem:** Pretend deadlock never happens. (Surprisingly, **UNIX** and Windows often use this because deadlocks are rare, and it is cheaper to just restart the PC).

5. Deadlock Prevention vs. Avoidance

It is important to know the difference between these two:

- **Prevention:** This means breaking the rules. You change the system so that one of the "Four Conditions" (explained above) becomes impossible.
  - *Example:* To break "Circular Wait," you force processes to request resources in a specific order (like 1, then 2, then 3).
- **Avoidance:** The system is smarter. It asks for information **before** it starts. It requires the process to say, "I might need a maximum of 5 printers." The OS then checks if giving a resource now will create a danger later.
  - **Safe State:** The system is "Safe" if there is a specific order to run all processes so everyone finishes.
  - If the system is **Safe**, there is **no deadlock**.
  - If the system is **Unsafe**, a deadlock is **possible** (but not 100% guaranteed).

6. The Banker's Algorithm (Slide 35)

This is a specific method for **Deadlock Avoidance** when there are multiple instances of resources.

- **The Name:** It is called the Banker's Algorithm because it works like a bank. A bank has a limited amount of cash (capital). It has many customers (processes) who want loans (resources).
- **The Rule:** The bank will only lend money if it knows it will still have enough cash to satisfy the needs of at least one customer, letting that customer finish and pay back the loan, so the money can be used for the next customer.
- **Data Structures (Important Math):**
    - **Available:** How much money/resources the bank has right now.
    - **Max:** The maximum loan a customer might ever need.
    - **Allocation:** How much the customer has borrowed so far.

Need: How much *more* the customer needs to finish. (
Need=Max−AllocationNeed = Max - AllocationNeed=Max−Allocation

    -
).
-

7. Detection and Recovery

If we don't prevent or avoid deadlocks, we must detect them.

- **Detection:** The system runs a check periodically. It looks at the graph. If it sees a cycle (and single instances), it knows there is a deadlock.
- **Recovery:** Once a deadlock is found, the OS must fix it. It usually does this by **terminating processes** (killing the program) or **preemption** (stealing the resource back).
    - *Problem with Recovery:* **Starvation**. If we always pick the same process to be the victim (the one that gets killed), that process will never finish its work. It starves.
-

Part 2: Key Points to Memorize

These are the definitions, lists, and rules you must memorize for the exam.

1. Definitions

- **Deadlock:** A state where every process in a set is waiting for an event that can be caused only by another process in the same set.
- **Safe State:** A state is safe if the system can allocate resources to each process (up to its maximum) in some order and still avoid deadlock.
- **Unsafe State:** A state that *may* lead to a deadlock.

2. The 4 Necessary Conditions (Remember all 4!)

Deadlock occurs only if **all** these hold simultaneously:

1. **Mutual Exclusion:** Only one process can use a resource at a time (non-sharable).
2. **Hold and Wait:** A process holding a resource is waiting to get others.
3. **No Preemption:** Resources cannot be taken forcefully; they must be released voluntarily.

Circular Wait: A closed chain of processes waiting for each other (
P0→P1→P2⋯→P0P_0 \rightarrow P_1 \rightarrow P_2 \dots \rightarrow P_0P0→P1→P2⋯→P0

4.
).

3. Resource Allocation Graph (RAG) Rules

- **No Cycle:** No deadlock.
- **Cycle (Single instance):** Deadlock exists.
- **Cycle (Multiple instances):** Deadlock is possible, but not certain.
- **Claim Edge (dashed line):** Shows that a process *may* request a resource in the future (Used in Avoidance).

4. Handling Methods Summary

- **Prevention:** Ensures at least **one** of the 4 necessary conditions cannot happen.
- **Avoidance:** Needs "a priori" (future) information about **maximum** resources. Uses **Banker's Algorithm**.
- **Detection:** Checks for circular waits using a Wait-For Graph.
- **Ignore:** Used by most OS (including UNIX/Windows). Assumes deadlocks are rare.

5. Banker's Algorithm Variables

You must know these matrices for calculation problems:

- **Available[j]:** Amount of resource j currently free.
- **Max[i,j]:** Maximum amount Process i will ever want of resource j.
- **Allocation[i,j]:** Amount Process i holds right now.
- **Need[i,j]:** Amount Process i still needs.
    - **Formula to memorize:** Need[i,j] = Max[i,j] - Allocation[i,j]
-

6. Safety Algorithm Logic (Steps)

To find if a system is Safe:

1. Look for a process whose Need is less than or equal to Available (Work).
2. If you find one, pretend it finishes. Take its Allocation and add it to Available.
3. Repeat until all processes are finished.

If everyone finishes
→\rightarrow→

4. **Safe State**.

If you get stuck
→\rightarrow→

5. **Unsafe State**.

7. Recovery Options

- **Process Termination:**
  - Abort all deadlocked processes (Expensive!).
  - Abort one by one until the cycle breaks.
-
- **Resource Preemption:**
  - Select a **Victim** (Process to take resources from).
  - **Rollback** (Restart process to a safe state).
  - Avoid **Starvation** (Don't pick the same victim every time).
-

Part 3: Final Summary

This chapter is about **Deadlocks**, which is when processes get stuck because they are waiting for each other's resources forever.

For a deadlock to happen, you absolutely need four conditions at the same time: **Mutual Exclusion** (sharing not allowed), **Hold and Wait** (keeping while waiting), **No Preemption** (no stealing resources), and **Circular Wait** (waiting in a circle).

We can visualize this using a **Resource Allocation Graph (RAG)**. In this graph, if you see a cycle (a circle in the arrows), there is a problem. If there is only one copy of the resource, a cycle guarantees a deadlock.

To fix or stop deadlocks, operating systems use three strategies:

1. **Prevention:** Change the rules so one of the 4 conditions is impossible (e.g., force processes to request resources in order to stop Circular Wait).
2. **Avoidance:** Use the **Banker's Algorithm**. The process tells the OS its **Maximum** needs beforehand. The OS calculates if granting a request is **Safe** or **Unsafe** (might lead to deadlock) based on what is **Available**, **Allocated**, and **Needed**.
3. **Detection and Recovery:** Let it happen, check the graph for cycles, and then recover. Recovery involves terminating (killing) processes or preempting (taking back) resources. Note that repeatedly taking resources from the same process causes **Starvation**.

Most real systems, like **UNIX**, simply ignore the problem because complex solutions slow the computer down too much. They leave it to the user to restart the program.

8

Part 1: Concepts to Understand

1. The Background of Memory

To run a program (like a game or a Word document), the computer must move it from the **Disk** (Hard Drive) into the **Main Memory** (RAM). The CPU (processor) can only access the **Main Memory** and its own **Registers** directly. It cannot talk to the Disk directly because the Disk is too slow.

The Speed Problem:

- **Registers** are inside the CPU and are very fast (takes 1 CPU cycle).
- **Main Memory** is outside the CPU and is slower (takes many cycles).
- Because the CPU is fast and Memory is slow, the CPU might have to wait (stall). To fix this, we put a **Cache** (fast memory) between the CPU and Main Memory.

Protection:
We must protect the Operating System (OS) and other users from crashing each other. We use two registers (like borders/fences):

1. **Base Register:** The starting address of the protected memory.
2. **Limit Register:** The size/length of the memory.
   The CPU checks: *Is the address inside these borders?* If yes, it is safe. If no, it is an error (trap).

2. Logical vs. Physical Address (Important!)

You must understand that the computer sees memory differently than the user (programmer) does.

- **Logical Address (Virtual Address):** This is the address generated by the **CPU**. It is what the program *thinks* the address is.
- **Physical Address:** This is the *real* address in the **Memory Unit**.
- **MMU (Memory-Management Unit):** This is a hardware device that acts like a translator. It changes the Logical Address into the Physical Address at "run time" (while the program is running).
    - *How MMU works (Simple version):* It takes the Logical Address and adds a value from a **Relocation Register** to get the Physical Address.
    - *Example:* If Logical Address is 346 and Relocation Register is 14000, the Real Physical Address = 14346.
-

3. Swapping

Sometimes, the Main Memory is full.
Swapping is a technique where the OS temporarily moves a process out of Main Memory and onto a **Backing Store** (a fast disk). Later, it moves it back into Memory to finish running.

- **Backing Store:** A fast disk large enough to keep copies of all memory images.
- **Roll out, roll in:** This happens with priority. If a high-priority job comes, the OS swaps out (removes) a low-priority job to make space. When the high-priority job finishes, the low-priority job swaps back in.
- **Problem:** Swapping takes time because transferring data to a disk is slow. Mobile phones (Android/iOS) typically do **not** support normal swapping because flash memory has a limited lifespan for writing. Instead, they just terminate (kill) apps if memory is low.

4. Contiguous Memory Allocation

"Contiguous" means "all together" or "side-by-side." In this method, a process must sit in one single block of memory. It cannot be broken into pieces.

How we divide the memory:

1. **Fixed Partitions:** Memory is divided into fixed box sizes (e.g., every box is 100K).
    - *Problem:* If you put a small job (20K) into a big box (100K), you waste 80K of space inside the box. This waste is called **Internal Fragmentation**.
2.
3. **Dynamic (Variable) Partitions:** Memory creates a "hole" (partition) exactly the size the process needs.
    - *Example:* Process needs 55K? We cut a slice of exactly 55K.
    - *Hole:* A block of free available memory.
    - *Problem:* As processes finish and leave, we get many small, scattered empty holes. If we have a new process that needs 100K, and we have two separate 50K holes, we cannot fit the process because it is not contiguous. This wasted scattered space is called **External Fragmentation**.
4.

How to choose a hole (Placement Algorithms):
If we have a list of empty holes (e.g., 10K, 20K, 5K, 30K), which one do we give to a new process?

1. **First-Fit:** Allocate the *first* hole that is big enough. (Advantage: Fast).
2. **Best-Fit:** Allocate the *smallest* hole that is big enough. (Advantage: Creates smallest leftovers. Disadvantage: Slow searching).
3. **Worst-Fit:** Allocate the *largest* hole available. (Disadvantage: Leaves big unused gaps).
   Result: First-fit and Best-fit are better than Worst-fit in terms of speed and storage use.

How to fix External Fragmentation?
We use **Compaction**.

- Imagine a room with messy chairs. Compaction is like pushing all the chairs to one side of the room so you have a big empty space on the other side.
- The OS shuffles memory to put all free memory together in one large block.

5. Non-Contiguous Allocation (The Better Way)

Since "Contiguous" causes fragmentation, modern systems use **Non-Contiguous** methods. This means a program can be broken into pieces and scattered around memory. There are two ways: Segmentation and Paging.

A. Segmentation
This divides memory based on the **user's view** (logical units). A program is not just a line of code; it has parts: a main program, functions, a stack, variables, etc.

- Each part becomes a **Segment**.
- Segments have different sizes.
- **Hardware:** We need a **Segment Table**. It stores:
    - **Base:** Starting physical address.
    - **Limit:** The length of the segment.
-
- To find an address, we check if the **offset** (location inside the segment) is less than the **limit**. If it is valid, we add the **base** + **offset**.

B. Paging (Very Important)
Paging cuts memory into fixed, equal-sized blocks. It does not care about logical functions.

- **Frames:** Fixed-size blocks of *Physical Memory*.
- **Pages:** Fixed-size blocks of *Logical Memory*.
- **Rule:** Page Size **must equal** Frame Size. (Power of 2, usually between 512 bytes and 8192 bytes).

How Paging Works:

1. The Logical Address is divided into two parts:
    - **p (Page Number):** Which page is it?
    - **d (Page Offset):** Where exactly inside that page?
2.
3. The OS uses a **Page Table** to look up p. The table tells the OS which physical **Frame (f)** holds that page.
4. The Physical Address is found by replacing p with f and keeping d.
    - *Formula:* Physical Address = (Frame Number × Frame Size) + Offset.
5.

Pros and Cons of Paging:

- **Advantage:** No **External Fragmentation**. We can put pages anywhere in memory. Allocating memory is easy (just find any free frame).
- **Disadvantage:** We might have **Internal Fragmentation**. If a program needs 305 bytes, and pages are 100 bytes, we need 4 pages (400 bytes). The last 95 bytes are wasted space inside the last page.

Part 2: Key Points to Memorize

Here are the rules and definitions you must memorize for the exam.

1. Definitions

- **Logical Address (Virtual):** Address generated by the CPU.
- **Physical Address:** Address seen by the memory unit (RAM).
- **MMU (Memory-Management Unit):** Hardware that maps Virtual to Physical addresses at run time.
- **Swapping:** Temporarily moving a process from memory to a backing store (disk) and back.
- **Backing Store:** Fast disk used for swapping.
- **Hole:** A block of available free memory.

2. Fragmentation Types

- **Internal Fragmentation:** Unused memory *inside* a partition. (Happens in Fixed Partitions and Paging).
    - *Cause:* Partition size > Process size.
-
- **External Fragmentation:** Total memory exists, but it is not contiguous (scattered). (Happens in Dynamic Partitions).
    - *Solution:* **Compaction** (shuffling memory to merge free space).
-

3. Allocation Algorithms (For Contiguous Memory)

- **First-fit:** Allocate the first hole that is big enough. (Fast).
- **Best-fit:** Allocate the smallest hole that is big enough. (Efficient).
- **Worst-fit:** Allocate the largest hole. (Produces largest leftovers).

4. Segmentation vs. Paging

- **Segmentation:**
    - Based on: **User View** (logical units like Main, Stack, Array).
    - Sizes: **Variable** (different sizes).
    - Hardware: Uses **Base** (Start) and **Limit** (Length).
    - Addresses: <Segment Number, Offset>.
-
- **Paging:**
    - Based on: **Fixed Blocks** (hardware view).
    - Sizes: **Fixed** (Page Size = Frame Size).
    - Hardware: Uses **Page Table**.
    - **Frames:** Physical memory blocks.
    - **Pages:** Logical memory blocks.
    - Addresses: <Page Number (p), Offset (d)>.
-

5. Protection & Hardware Bits

- **Relocation Register:** Used for dynamic linking/address translation.
- **Valid/Invalid Bit (in Paging):**
  - **Valid (v):** The page is in the process's logical address space (legal).
  - **Invalid (i):** The page is not in the address space (illegal/trap).
-
- **PTBR:** Page-Table Base Register (points to the page table).
- **PTLR:** Page-Table Length Register (indicates size of the page table).

6. Math Formulas (Important for Calculations)

Physical Address Calculation (Paging):

Address=(FrameNumber×FrameSize)+OffsetAddress = (FrameNumber \times FrameSize) + OffsetAddress=(FrameNumber×FrameSize)+Offset

-
- **Address Spaces:**

If Logical Address Space is
2m2^m2m

and Page Size is
2n2^n2n

  -
    :

Page Number (
ppp

):
m−nm - nm−n

    -
      bits.

Page Offset (
ddd

):
nnn

    -
      bits.
-

Total Pages Formula:

NumberofPages=Total Logical MemoryPage SizeNumber of Pages = \frac{\text{Total Logical Memory}}{\text{Page Size}}NumberofPages=Page SizeTotal Logical Memory

Part 3: Final Summary

Chapter 8 explains how the computer manages its limited RAM. The CPU creates **Logical Addresses**, but the memory hardware uses **Physical Addresses**. A hardware device called the **MMU** translates these addresses using relocation registers.

We tried allocating memory in contiguous blocks (**Contiguous Allocation**), either using fixed slots or dynamic slots.

- **Fixed Partitions** cause **Internal Fragmentation** (wasted space inside the box).
- **Dynamic Partitions** cause **External Fragmentation** (wasted space between boxes).
- We can fix external fragmentation using **Compaction**, but it is slow.

Because contiguous allocation is inefficient, modern systems use **Non-Contiguous Allocation**, which splits programs into pieces so they can be scattered in memory.

1. **Segmentation** divides the program by logical parts (code, data, stack) of different sizes.
2. **Paging** is the most common method. It divides:
    - **Physical Memory** into fixed blocks called **Frames**.
    - **Logical Memory** into fixed blocks called **Pages**.
    - A **Page Table** maps pages to frames.
3.

Paging is superior because it completely solves **External Fragmentation**. Although it causes a small amount of Internal Fragmentation in the final page of a process, it makes allocating memory extremely efficient because any free frame can be used for any page. The OS uses hardware bits (Valid/Invalid) to ensure memory protection.

9

Part 1: Concepts to Understand

1. What is Virtual Memory?

In previous chapters, we learned that a program must be in the "Physical Memory" (RAM) to run. But, Physical Memory is small and expensive. Hard Disks (Storage) are huge and cheap.

Virtual Memory is a technique that makes the computer appear to have much more memory than it actually does.

- **The Big Idea:** The programmer sees a huge "Logical Memory" (Virtual Memory). The computer only has a small "Physical Memory" (RAM).
- **How it works:** The Operating System (OS) separates the *Logical* memory from the *Physical* memory. It keeps only the **necessary parts** of the program in the Physical Memory. The rest of the program stays on the Hard Disk (Backing Store) until it is needed.

Why do we use it?

1. We can run programs that are **larger** than the actual Physical Memory.
2. We can run **more programs** at the same time (because each program takes less space in RAM).
3. We need **less I/O** (loading/saving data) to start a program, so programs run faster.

2. Demand Paging (The "Lazy" Swapper)

Imagine you are at a restaurant. You do not order every single item on the menu at once. You only order the dish you want to eat right now.

Demand Paging is exactly like that. It uses a "Lazy Swapper."

- When a program starts, the OS **does not** load the whole program into memory.
- It brings a **page** (a piece of the program) into memory **only when it is needed** for execution.
- If a page is never used, it is never loaded.

Valid-Invalid Bit:
How does the computer know if a page is in Memory (RAM) or on the Disk?

- It uses a "Valid-Invalid Bit" in the Page Table.
- **v (Valid):** The page is currently in RAM. The CPU can use it.
- **i (Invalid):** The page is NOT in RAM. It is either on the disk or does not exist. If the CPU tries to use a page marked **i**, it causes an error called a **Page Fault**.

3. What happens during a Page Fault?

A **Page Fault** is a specific interrupt that happens when a program tries to access a page that is marked "Invalid" (not in memory).

The 6 Steps of Handling a Page Fault (Slide 16):

1. **Check:** The OS looks at an internal table to see if the reference was valid.
2. **Trap:** If the reference was invalid, the program is stopped. If it was valid but just not in memory, we must load it.
3. **Find Space:** The OS finds a **Free Frame** (empty space) in the Physical Memory.
4. **Disk Operation:** The OS reads the data from the Disk and puts it into the Free Frame.
5. **Update:** The OS updates the Page Table. It sets the bit to **v (valid)**.
6. **Restart:** The OS restarts the instruction that caused the trap. Now the data is there, so it succeeds.

4. Page Replacement

What happens if the Physical Memory is 100% full? There are no free frames.
To bring in a new page, we must kick out an old page. This is called **Page Replacement**.

To do this efficiently, we use a **Modify Bit (Dirty Bit)**.

- If a page was modified (changed) while it was in memory, we must write it back to the disk (save it) before we delete it.
- If it was **not** modified, we can just overwrite it without saving (because the copy on the disk is still the same). This saves time.

Basic Algorithm for Replacement:

1. Find the location of the desired page on the disk.
2. Find a **Victim Frame** (a page currently in memory to kick out).
3. Write the Victim Frame to the disk (if necessary).
4. Read the new page into the empty frame.
5. Update the tables and restart the process.

5. Page Replacement Algorithms

We need a rule to decide **which** page to kick out. The goal is to have the **minimum number of page faults**.

A. FIFO (First-In, First-Out):

- **Rule:** Kick out the page that has been in memory the **longest** (the oldest one).
- **Problem:** The oldest page might be very important and used often.
- **Belady's Anomaly:** Usually, giving a program more memory frames means fewer errors. But with FIFO, sometimes adding more frames actually causes **more** page faults. This bad behavior is called Belady's Anomaly.

B. OPT (Optimal Algorithm):

- **Rule:** Replace the page that will **not be used for the longest period of time**.
- **Problem:** We cannot predict the future! We do not know what the user will click next.
- **Use:** This is impossible to implement in real life. We only use it as a benchmark to compare other algorithms against.

C. LRU (Least Recently Used):

- **Rule:** Replace the page that **has not been used for the longest time**. (Look backward at the past).
- **Why:** If you haven't used a page in a long time, you probably won't use it again soon.
- **Implementation:** This is a good algorithm, but it is hard to build. You need special hardware (Counters or Stacks) to keep track of the time.

6. Thrashing

Thrashing is a very bad situation where the system spends more time moving pages (swapping in and out) than actually executing the program.

- **Cause:** This happens if the CPU utilization is low, so the OS adds more programs (multiprogramming). However, there is not enough total memory for all of them. The total need for frames is higher than the available frames.
- **Result:** The programs get stuck waiting for pages. The CPU usage drops. The OS adds *more* programs to try to help, but it makes the traffic jam worse. The system "freezes."

How to stop Thrashing?
We use the **Working-Set Model**.

- The "Working Set" is the group of pages a process is using *right now* (Locality).
- If we ensure every process has enough frames to hold its current Working Set, thrashing will not happen.

Part 2: Key Points to Memorize

Memorize these definitions, steps, and rules for the final exam.

1. Definitions

- **Virtual Memory:** Separation of user logical memory from physical memory. It allows execution of processes that are not completely in memory.
- **Demand Paging:** A system where pages are loaded into memory only when they are needed during execution (Lazy Swapper).
- **Page Fault:** An interrupt that occurs when a program attempts to access a page that is marked invalid (not currently in RAM).
- **Lazy Swapper:** A swapper that never swaps a page into memory unless that page is explicitly needed.
- **Pager:** A swapper that deals with individual pages of a process rather than the entire process.
- **Belady’s Anomaly:** A phenomenon where increasing the number of page frames results in an increase in the number of page faults (happens in FIFO).
- **Thrashing:** A state where a process spends more time paging (swapping pages in/out) than executing code.

2. Valid vs. Invalid Bit

vvv

- **(valid):** The page is in physical memory (RAM).

iii

- **(invalid):** The page is NOT in memory (it is on the disk or invalid).

3. Page Fault Steps (Important for Exam)

You must be able to list the logic of handling a fault:

Reference: Check table. If invalid
→\rightarrow→

Abort. If valid but not in memory
→\rightarrow→

1. proceed.
2. **Trap:** Trap to the OS (Operating System).
3. **Frame:** Find a free frame (or select a victim).
4. **Swap:** Read the page from the disk into the frame.

Reset: Update the page table (set bit to
vvv

5.).
6. **Restart:** Restart the instruction.

4. Page Replacement Algorithms Comparison

- **FIFO (First-In, First-Out):**
  - Replaces the oldest page.
  - Suffers from **Belady's Anomaly**.
  - Easy to program but performs poorly.
-
- **OPT (Optimal):**
  - Replaces the page that will not be used for the longest time (future).
  - Lowest possible page-fault rate.
  - Impossible to implement (requires future knowledge).
-
- **LRU (Least Recently Used):**
  - Replaces the page not used for the longest time (past).
  - Good performance (approximation of OPT).
  - Implementation requires **Counters** or **Stacks**.
-

5. Frame Allocation Types

- **Local Replacement:** A process selects a victim frame only from its *own* set of allocated frames.
- **Global Replacement:** A process selects a victim frame from *all* frames in the system (it can take a frame from another process).
    - *Note:* Global replacement usually provides better system throughput and is more common.
-

6. Working Set Model (Avoiding Thrashing)

- **Locality:** Pages are usually referenced in groups together.

Δ\DeltaΔ

-

(Delta): The Working-Set Window (a fixed number of page references, e.g., 10,000 instructions).

WS(t1)WS(t_1)WS(t1)

: The set of pages referenced in the most recent
Δ\DeltaΔ

-

time.

If the sum of all working sets > Total Physical Memory
→\rightarrow→

-

Thrashing occurs.

Part 3: Final Summary

Chapter 9 covers **Virtual Memory**, a method that lets computers run large programs even if they have small RAM. It does this by tricking the program into seeing a large logical memory, while the OS actually manages a smaller physical memory.

The core technique is **Demand Paging**, where pages are only loaded from the disk when the program asks for them (using a **Lazy Swapper**). A **Valid/Invalid bit** in the Page Table tells the hardware if a page is in memory (

vvv

) or on the disk (
iii

). If the CPU asks for a page marked
iii

, a **Page Fault** occurs. The OS pauses the program, loads the data from the disk to a free frame, updates the table, and restarts.

If memory is full, the OS must choose a "Victim Frame" to delete to make space. This is **Page Replacement**. We use algorithms like **FIFO** (remove oldest), **OPT** (remove future unused), and **LRU** (remove least recently used) to minimize the number of page faults. FIFO is simple but has a flaw called **Belady's Anomaly**. OPT is the best but impossible to build. **LRU** is the most practical good choice.

Finally, if the system tries to do too much at once and memory fills up completely, processes spend all their time waiting for pages and no time working. This is called **Thrashing**. We fix this by ensuring every process has enough memory for its **Working Set** (the pages it is currently using).

10

Part 1: Concepts to Understand

1. What is a File?

In the physical world, if you have a lot of papers, you put them in a folder. In a computer, we use **Secondary Storage** (like Hard Disks or DVDs) to keep data forever. But disks are just machines with magnetic platters. We need a better way to organize data.

The Operating System (OS) creates a "logical view." This means it hides the messy details of the hard disk and presents you with a clean **File**.

- **Definition:** A file is a named collection of related information recorded on storage.
- **The User's View:** A file has a name (like homework.doc), a type (text, picture, program), and a size.
- **The System's View:** The system maps this file onto the physical device (hard disk blocks).

File Attributes (The File's ID Card):
Just like a person has an ID card with their name and address, a file has **Attributes**. These are stored in the **Directory**:

- **Name:** The only part a human reads.
- **Identifier:** A unique number the computer uses to tell files apart.
- **Type:** Helps the system know if it is a text file, a music file, etc.
- **Location:** Where is it on the hard disk?
- **Size:** How big is it?
- **Protection:** Who can read it? Who can write on it?
- **Time & Date:** When was it created or changed?

2. Operations: What can we do with a File?

The OS provides specific tools (system calls) to manage files:

1. **Create:** Make a new empty file.
2. **Write:** Put data into the file. (We use a "write pointer" to know where to write).
3. **Read:** Look at the data. (We use a "read pointer" to know where we are reading).
4. **Reposition (Seek):** Jump to a specific part of the file without reading everything before it.
5. **Delete:** Remove the file and free up space.
6. **Truncate:** Keep the file but erase all the contents inside it (size becomes 0).

Important Note on Open/Close:

- **Open(Fi):** Before using a file, you must open it. The OS searches the directory, finds the file, and moves its details (inode) into the active memory table.
- **Close(Fi):** When you finish, you close it. The OS removes it from memory and saves necessary info back to the disk.

3. Access Methods: How do we read the data?

Imagine you have a book. There are two ways to read it:

1. **Sequential Access (Like a Cassette Tape):** You start at the beginning and go forward page by page. If you want to get to page 100, you must pass pages 1 through 99.
    - *Usage:* This is the most common method (editors, compilers).
2.
3. **Direct Access (Like a Music CD or Hard Disk):** You can jump directly to any track (or block) instantly. You do not need to play the previous songs.
    - The file is viewed as numbered blocks (0, 1, 2...). You say "Read Block 14," and it jumps there.
    - *Usage:* Very useful for databases where you need to find specific information quickly.
4.

4. Directory Structure

A disk can be huge. To keep files organized, we split the disk into **Partitions** (or Volumes). Inside these volumes, we need a map to tell us where files are. This map is the **Directory**.

We can organize the directory in different ways:

A. Single-Level Directory:

- **Concept:** One big folder for everyone.
- **Problem:** If you and I both want to create a file named "Test", we cannot. The names must be unique. It is hard to group files.

B. Two-Level Directory:

- **Concept:** Every user gets their own directory (User 1, User 2...).
- **Advantage:** Different users can use the same file name.
- **Disadvantage:** You still cannot group your own files (like separating games from homework).

C. Tree-Structured Directory (This is what Windows/Mac uses):

- **Concept:** There is a **Root** directory (/ or C:). Inside it, you can have files or subdirectories.
- **Advantage:** Very efficient. You can group files logically. You can search easily.
- **Path Names:**
    - **Absolute:** Starts from the root (e.g., /home/user/music/song.mp3).
    - **Relative:** Starts from where you are right now (e.g., music/song.mp3).
-

D. Acyclic-Graph Directory (Shared Files):

- **Concept:** This looks like a tree, but it allows **Sharing**. A file can appear in two different folders at once. It is not a copy; it is the *same* file linked in two places.
- **Aliasing:** The file has different paths to reach it.
- **Implementation:** We use **Links** (pointers).
- **Problem:** If User A deletes the file, does it disappear for User B?
    - *Dangling Pointer:* A pointer that points to nothing (bad).
    - *Solution:* Only delete the file when all references (users pointing to it) are gone.
-

5. File Protection

Because computers have many users, we must control who accesses files.
We control three main actions: **Read**, **Write**, and **Execute**.

Method 1: Access Control Matrix
Imagine a giant grid.

- Rows = Users.
- Columns = Files.
- In the boxes, we write what rights the user has.
- *Problem:* The matrix is too big and mostly empty (sparse). It wastes space.

Method 2: Access Control List (ACL)
Instead of a giant grid, each **File** carries a list. The list says: "User A can Read, User B can Write."

- This uses less space but can be tedious to manage.

Method 3: UNIX/Linux Style (Condensed)
To save space, UNIX groups users into three categories:

1. **Owner:** The person who created the file.
2. **Group:** A team of users who share work.
3. **World (Public):** Everyone else.

We assign 3 bits (rwx) to each category:

- **r (read) = 4**
- **w (write) = 2**
- **x (execute) = 1**

If you want the Owner to have full access (Read+Write+Execute), you add

4+2+1=74+2+1 = 74+2+1=7

.
If the Group can only Read, you give them
444

.
The code would be **740** (Owner=7, Group=4, World=0).

Part 2: Key Points to Memorize

1. Key Definitions

- **File:** A named collection of related information stored on secondary storage. It is the smallest unit of storage the OS manages.
- **Directory:** A symbol table that translates file names into directory entries (location on disk).
- **Volume:** Any entity containing a file system.
- **Extension:** The part of the name after the dot (e.g., .docx, .exe) that identifies the file type.
- **Relative Path:** Defines a path starting from the **current working directory**.
- **Absolute Path:** Defines a path starting from the **root directory**.

2. Access Methods Comparison

Method	Description	Advantages
Sequential	Read data in order (byte by byte). Used by tape drives.	Simple. Good for compilers/editors.
Direct	Jump to specific blocks (numbers) randomly. Used by HDDs.	Fast access to random parts. Good for databases.
Indexed	Use an index (like a book's index) to find the location pointer.	Faster searching for huge files.

3. Protection Mechanisms (UNIX chmod)

You must be able to calculate permission numbers.

- **Bits:** r (read) = 4, w (write) = 2, x (execute) = 1.
- **Structure:** [Owner] [Group] [Public/World]
- **Example:** A file allows the **Owner** to Read/Write, **Group** to Read, **World** to have no access.
    - Owner: r+w = 4+2 = **6**
    - Group: r = **4**
    - World: None = **0**
    - **Result:** 640.
-

4. Directory Structures

- **Single-Level:** No grouping, naming problems (collisions).
- **Tree-Structured:** Efficient searching, allows grouping (subdirectories), current standard.
- **Acyclic-Graph:** Allows **shared** files/directories (links).
- **Garbage Collection:** Required in General Graph directories to determine when a file can be safely deleted (no loops).

5. Disk Structure Terms

- **Platters:** The circular disks inside a hard drive.
- **Heads:** The reading/writing arms that move across the disk.
- **Tracks:** Circles on the disk where data is stored.

Part 3: Final Summary

Chapter 10 introduces the **File System**, which is the Operating System's way of storing and organizing data on storage devices (like Hard Disks). A **File** is an abstract idea; physically, it is just a collection of bits on a magnetic disk.

We organize files using **Directories**.

- Early systems used **Single-Level** directories (one list for everyone), which caused naming conflicts.
- Modern systems use **Tree-Structured** directories (folders inside folders). This allows logical grouping and unique paths for every file.
- We can access files **Sequentially** (reading in order) or via **Direct Access** (jumping to specific blocks).

Protection is vital. We cannot let every user delete system files. We implement **Access Control**.

- **ACL (Access Control Lists)** tell us exactly which user can do what to a file.
- **UNIX Protection** simplifies this into three groups: **Owner, Group,** and **Public**, using numbers (Read=4, Write=2, Execute=1) to define permissions.

When files are shared (Acyclic Graph directories), we use **Links**. This creates an issue: if the original file is deleted, the link becomes a "dangling pointer" pointing to nothing. The OS must handle this carefully.

Part 1: Concepts to Understand

This chapter explains **how** the Operating System (OS) actually stores files on the hard disk. In the previous chapter, we talked about what a file is. Now, we talk about how the engineers build the system to organize these files on the physical disk blocks.

1. File-System Structure (Layers)

The file system is complex, so it is organized into **layers** (levels). Think of it like a company where the boss gives an order, managers process it, and workers do the physical job.

Here are the layers from top to bottom (Slides 6-7):

1. **Logical File System (The Manager):** It manages the metadata. **Metadata** means "data about data"—like who owns the file, permissions, and file names. It knows the file structure but does not touch the actual hard disk.
2. **File-Organization Module (The Translator):** It translates the logical addresses (0, 1, 2...) used by the logical system into physical addresses on the hard drive. It also tracks free space.
3. **Basic File System:** It issues generic commands to the device drivers, like "Read block 5" or "Write block 10."
4. **I/O Control:** These are the **Device Drivers**. They speak the specific language of the hardware to control the actual physical parts of the disk.

2. Allocation Methods (How to put files on disk)

This is the most important part of the chapter. A hard disk is a collection of "blocks" (storage spaces). When we create a file, we need to assign blocks to it. There are three main ways to do this:

Method A: Contiguous Allocation (The "Together" Method)

- **Idea:** Each file occupies a set of blocks that are right next to each other (contiguous).
- **Example:** If a file is 3 blocks long and starts at block 6, it will occupy blocks 6, 7, and 8.
- **Good:** It is **simple** (you only need the start address and length). It is very **fast** to read because the head of the disk doesn't need to move much.
- **Bad:**
    - **Finding Space:** It is hard to find a big enough empty space for a large file.
    - **External Fragmentation:** As files are deleted, you get small empty gaps (holes) that are too small to use.
    - **File Growth:** If a file needs to grow, but there is another file right next to it, it cannot grow. You must move the whole file to a bigger empty spot.

Method B: Linked Allocation (The "Treasure Hunt" Method)

- **Idea:** Each file is a linked list. Each block contains data and a **pointer** (arrow) to the next block's address. The blocks can be scattered anywhere on the disk.
- **Good:**
    - **No Fragmentation:** You can use any free block anywhere.
    - **Easy growth:** If the file grows, just pick any empty block and point to it.
-
- **Bad:**
    - **Slow Random Access:** If you want to read the 50th block, you cannot jump there directly. You must start at block 1, read the pointer to block 2, then block 3... all the way to 50.
    - **Reliability:** If one pointer breaks or gets corrupted, the rest of the file is lost.
-
- **Improvement (FAT):** Microsoft's **FAT (File Allocation Table)** is a version of this. Instead of pointers being inside the blocks, there is a special table at the beginning of the disk that stores all the links. This is faster because the table can be loaded into memory.

Method C: Indexed Allocation (The "Table of Contents" Method)

- **Idea:** All pointers for a file are gathered together in one special block called the **Index Block**.
- **Analogy:** Think of the index block like a book's table of contents. It lists "Page 1 is at block 10, Page 2 is at block 55..."
- **Good:** Supports **Direct Access**. You can calculate exactly where block 50 is by looking at the index. No external fragmentation.
- **Bad:**
    - **Overhead:** You "waste" a whole block just to store pointers (the index block), even for small files.
    - **Size Limits:** If a file is huge, one index block might not hold enough pointers.
-
- **Solution for Huge Files:** We use "Multilevel Indexes" or "Combined Schemes" (like in **UNIX inodes**). This is where an index block points to *another* index block, allowing for very large files.

3. Free-Space Management

How does the OS know which blocks are empty and safe to write on? It maintains a **Free-Space List**.

There are four ways to manage free space (Slide 32):

1. **Bit Map (Bit Vector):** A list of bits (0s and 1s). 0 means used, 1 means free. It is simple and easy to find the first free block, but it uses memory.
2. **Linked List:** All free blocks are linked together. This saves memory space because you don't need a separate list, but it is slow to traverse.

Grouping: Stores the addresses of
nnn

3.
free blocks inside the first free block.

4. **Counting:** Stores the address of the first free block and the number (count) of contiguous free blocks following it. This is good when space is usually allocated in chunks.

4. Recovery

Sometimes, computer power fails or the system crashes. This can corrupt the file system.

- **Consistency Checking:** A system tool (like fsck or chkdsk) compares the data in the directory with the data on the disk. If they don't match, it tries to fix the errors. This process is slow.
- **Backups:** This is the only way to be 100% safe. You use system programs to copy data from the disk to another storage device (like a separate hard drive or tape). If the original disk fails, you restore data from the backup.

Part 2: Key Points to Memorize

Here are the definitions and facts you must write in the exam.

1. Layers of File System

Memorize the layers in order from Top (Software) to Bottom (Hardware):

1. **Logical File System:** Manages metadata.
2. **File-Organization Module:** Maps logical block numbers to physical blocks; manages free space.
3. **Basic File System:** Issues commands to drivers.
4. **I/O Control:** Device drivers and interrupt handlers.

2. Allocation Methods Summary

- **Contiguous Allocation:**
  - **Definition:** Each file occupies a set of contiguous blocks on the disk.
  - **Pros:** Best performance (speed), simple.
  - **Cons:** Finding space is hard, external fragmentation, file growth is difficult.
  - **Variation:** Extent-based systems (allocates chunks called "extents" instead of single blocks).
- **Linked Allocation:**
  - **Definition:** Each file is a linked list of blocks. Each block has a pointer to the next block.
  - **Pros:** No external fragmentation, easy file extension.
  - **Cons:** No direct access (slow for random access), space wasted for pointers, reliability (lost pointers).
  - **Example:** FAT (File Allocation Table) used by MS-DOS/Windows uses a table for links.
- **Indexed Allocation:**
  - **Definition:** Brings all pointers together into one **Index Block**.
  - **Pros:** Supports direct access, no external fragmentation.
  - **Cons:** Overhead (wasted space) of index blocks.
  - **Handling Large Files:** Use **Linked Scheme**, **Multilevel Index** (2-level), or **Combined Scheme** (Unix inodes: Direct + Single Indirect + Double Indirect).

3. Free-Space Management Methods

1. **Bit Map (Bit Vector):** Uses 1 bit per block.
    - 1 = Free
    - 0 = Used
    - Calculation: Block number = (number of bits per word) * (number of 0-value words) + offset of first 1 bit.
2.
3. **Linked List:** Connects all free blocks together. Slow to navigate.

Grouping: First free block stores addresses of
nnn

4.
other free blocks.
5. **Counting:** Stores the first free block address and the **count** of contiguous free blocks.

4. Recovery

- **Consistency Checking:** Compares directory structure with data blocks to fix logical errors. Can be slow.
- **Backup:** Using system programs to copy data to external storage (like tapes) for restoration if the disk fails.

5. Definitions

- **Volume:** Any entity containing a file system.
- **Metadata:** Information about a file (name, size, permissions), managed by the Logical File System.
- **Boot Control Block:** Information needed by the system to boot (start up).
- **Volume Control Block:** Information about the volume (number of blocks, block size).
- **File Control Block (FCB):** A storage structure consisting of information about a specific file.

Part 3: Final Summary

Chapter 11: File-System Implementation focuses on how the operating system internally manages files on the disk.

The system is organized into layers: the **Logical File System** (metadata), the **File-Organization Module** (translation), the **Basic File System** (commands), and **I/O Control** (device drivers).

The biggest challenge is **Allocation Methods**:

1. **Contiguous:** Files are stored in one piece side-by-side. It is the fastest but suffers from **Fragmentation** and makes expanding files hard.
2. **Linked:** Files are scattered pieces connected by pointers. It solves fragmentation and expansion but is slow for random access.
3. **Indexed:** A special **Index Block** keeps a list of pointers to all data blocks. This solves fragmentation and supports fast access, but wastes some space for the index itself. For very large files, complex schemes like **Multilevel Indexes** or the **UNIX Inode Combined Scheme** are used.

To track which parts of the disk are empty, the OS uses **Free-Space Management**, commonly using a **Bit Map** (a list of bits) or a **Linked List**.

Finally, systems need **Recovery** methods like consistency checkers (to fix logical errors after a crash) and backups (to recover data after physical failure).

Part 1: Concepts to Understand

This chapter explains the physical hardware used to store data (like Hard Drives) and how the Operating System (OS) manages them to make them fast and safe.

1. The Storage Hierarchy

Computers use different types of storage arranged like a pyramid (hierarchy) based on **speed**, **size**, and **cost** (Slide 3).

- **Top of the Pyramid (Fastest, Smallest, Expensive):**
  - **Registers:** Located inside the CPU.
  - **Cache:** Very fast memory.
  - **Main Memory (RAM):** Where active programs live.
-
- **Bottom of the Pyramid (Slowest, Largest, Cheapest):**
  - **Non-Volatile Memory (NVM/SSD):** Electrical storage.
  - **Hard Disk Drives (HDD):** Magnetic storage (spinning disks).
  - **Optical Disks/Magnetic Tapes:** Used for backups.
-

2. How a Hard Disk (HDD) Works

A hard disk works a bit like an old record player.

- **Platters:** These are round, flat disks that hold the data. They spin around very fast (like a fan).
- **Heads:** There is a reading/writing "Head" that hovers just above the platter.
- **Arm:** The Head is attached to an "Arm" that moves back and forth to find the right data.

Physical Structure:

- The disk surface is divided into rings called **Tracks**.
- Tracks are chopped into small slices called **Sectors**.
- A set of the same track on all platters stacked on top of each other is called a **Cylinder**.

Why is it slow?
Because the disk is mechanical (it moves), the computer has to wait for two things:

1. **Seek Time:** The time it takes to move the **Arm** to the correct **Cylinder**.
2. **Rotational Latency:** The time it takes for the **Disk** to spin so the correct **Sector** is under the head.

- **Note:** Solid-State Disks (SSD) use electricity (chips), not moving parts. They are much faster and more reliable but more expensive.

3. Disk Scheduling (Very Important)

The Operating System receives many requests to read or write files. Because moving the Disk Arm takes time (Seek Time), the OS must decide the **best order** to handle these requests to save time. This is called **Disk Scheduling**.

Here are the main algorithms (rules) for moving the disk arm:

- **FCFS (First-Come, First-Served):**
  - **Concept:** This is the simplest method. We serve requests in the exact order they arrive.
  - **Problem:** It is not efficient. The arm might jump wildly from one end of the disk to the other and back again.
-
- **SSTF (Shortest Seek Time First):**
  - **Concept:** Look at all the requests in the waiting line. Go to the one that is **closest** to where the arm is right now.
  - **Problem:** It can cause **Starvation**. If new requests keep arriving near the current position, a request far away might never get served.
-
- **SCAN (The Elevator Algorithm):**
  - **Concept:** The arm moves all the way to one end of the disk, and then reverses and moves all the way to the other end. It serves requests along the way, just like an elevator picking up people.
  - **Details:** It has a uniform wait time.
-
- **C-SCAN (Circular SCAN):**
  - **Concept:** Like SCAN, the arm moves from one end to the other, serving requests. BUT, when it hits the end, it immediately **jumps back** to the beginning *without* serving anyone on the return trip.
  - **Benefit:** This provides a more "uniform" wait time than SCAN because the return trip is essentially empty (very fast).
-
- **LOOK and C-LOOK:**
  - **Concept:** These are smarter versions of SCAN and C-SCAN. The arm goes only as far as the **last request** in that direction, then reverses. It does **not** go all the way to the physical end of the disk if no one needs data there.
-

4. RAID Structure (Making Disks Safer and Faster)

One disk can break (fail). If it breaks, you lose your data. To solve this, we connect multiple disks together to work as a team. This is called **RAID** (Redundant Array of Independent Disks).

RAID has two main goals:

1. **Reliability:** Keeping data safe if a disk fails (Redundancy).
2. **Performance:** Reading/Writing data faster.

Common techniques used in RAID:

- **Data Striping:** Taking a file, breaking it into pieces (blocks), and spreading those pieces across multiple disks.
    - *Benefit:* Speed. You can read/write different parts of the file from different disks at the same time (parallel).
-
- **Mirroring (Shadowing):** Keeping a perfect duplicate copy of every disk.
    - *Benefit:* High Reliability. If Disk 1 fails, the system instantly switches to Disk 2.
-
- **Parity:** A way to calculate missing data. It adds a "checksum" bit. If a drive fails, we can use the remaining data plus the Parity bit to calculate what the lost data was.

RAID Levels (Different Team Setups):

- **Level 0 (Striping):** Uses only striping. Very fast, but **no** reliability. If one disk fails, all data is lost.
- **Level 1 (Mirroring):** Duplicates everything. Very safe, but expensive (you need double the disks).
- **Level 2 & 3:** Use complex codes (Hamming code) and bit-level striping. Not commonly used now.
- **Level 4:** Uses striping for data, but keeps all Parity (safety data) on one specific disk. The Parity disk becomes a "bottleneck" (it gets overworked).
- **Level 5 (Distributed Parity):** Very popular. It stripes data **and** distributes the Parity across all disks. It is fast and safe.
- **Level 6 (P + Q Parity):** Like Level 5, but stores **two** different parity calculations. It can survive **two** disk failures at the same time.

Part 2: Key Points to Memorize

Here are the specific definitions, formulas, and lists you need to memorize for the exam questions.

1. Key Definitions

- **Seek Time:** The time it takes for the disk arm to move to the requested cylinder.
- **Rotational Latency:** The time it takes for the desired sector to rotate to the disk head.
- **Transfer Rate:** The speed at which data moves between the drive and the computer.
- **Positioning Time (Random-Access Time):** Seek Time + Rotational Latency.
- **Head Crash:** When the read/write head touches the disk surface (a physical failure).
- **Mean Time Between Failures (MTBF):** A measure of reliability; how long a disk typically lasts before breaking.

2. Disk Scheduling Algorithms

Memorize the behavior of each:

- **FCFS:** Orders requests by arrival time. Fair but slow.
- **SSTF:** Selects the request with the minimum seek time from current position. Risk of **starvation**.

SCAN: Moves like an elevator (Start
→\rightarrow→

End, Reverse, Start
←\leftarrow←

- End). Services requests in both directions.
- **C-SCAN:** Circular List. Services requests in **only one** direction. Jumps back to the beginning without servicing.
- **LOOK / C-LOOK:** Same as SCAN/C-SCAN but only goes as far as the last request, not to the disk edge.

3. RAID Concepts

- **Stripe Width:** The number of drives in the array.
- **Mirroring:** Duplication of disks (100% redundancy).
- **RAID 0:** Performance only. Uses **Striping**. No redundancy.
- **RAID 1:** Reliability focused. Uses **Mirroring**. Expensive.
- **RAID 4:** Uses Block Striping + a Dedicated Parity Disk.
- **RAID 5:** Uses Block Striping + **Distributed Parity**. Solves the bottleneck problem of RAID 4.
- **RAID 6:** Uses Block Striping + **Double Distributed Parity** (P + Q redundancy).

4. Storage Attachments

- **Host-Attached:** Storage connected via I/O ports (USB, SATA, SCSI).
- **Network-Attached Storage (NAS):** Storage accessed over a network connection (like Ethernet).
- **Storage-Area Network (SAN):** A private network connecting servers and storage units together.

5. Performance Calculation (Important Formula)

When calculating average access time for a specific file transfer:

AverageAccessTime=AverageSeekTime+AverageLatencyAverageAccessTime = AverageSeekTime + AverageLatencyAverageAccessTime=AverageSeekTime+AverageLatency

Average I/O Time=Average Access Time+(Amount to TransferTransfer Rate)+Controller Overhead\text{Average I/O Time} = \text{Average Access Time} + \left(\frac{\text{Amount to Transfer}}{\text{Transfer Rate}} \right) + \text{Controller Overhead}Average I/O Time=Average Access Time+(Transfer RateAmount to Transfer)+Controller Overhead

Note for calculation:
Average Latency=12×(60RPM)\text{Average Latency} = \frac{1}{2} \times \left(\frac{60}{\text{RPM}} \right)Average Latency=21×(RPM60)

-

Part 3: Final Summary

Chapter 12 covers the hardware that stores your data (Mass-Storage).

The most common device is the **Magnetic Hard Disk (HDD)**. It has spinning platters and a moving arm. The speed of the disk depends on how fast the arm can find the track (**Seek Time**) and how fast the disk spins to the sector (**Rotational Latency**). To make this faster, the Operating System uses **Scheduling Algorithms** (like FCFS, SSTF, SCAN, C-SCAN, and LOOK) to organize the order of requests.

FCFS is fair but slow. **SSTF** is fast but might ignore far-away requests (starvation). **SCAN/C-SCAN** move back and forth across the disk to serve everyone efficiently, like an elevator.

Because disks can fail, we use **RAID** technology to combine multiple disks.

- **RAID 0 (Striping)** makes the system fast by spreading data out.
- **RAID 1 (Mirroring)** makes the system safe by copying data.
- **RAID 5** balances speed and safety by using distributed parity codes, so if one disk breaks, the data can be recovered.

For exams, usually, you will be asked to calculate the head movement for different algorithms (FCFS vs SSTF vs SCAN) or to explain the difference between RAID 0, 1, and 5.

Final Praparation

THE "MAGICIAN PAPER": FINAL EXAM SURVIVAL GUIDE

Target Audience: Absolute Beginner (0% Knowledge)
Scope: Operating Systems Chapters 7 – 12
Language: Simple, Direct, Exam-Focused

MODULE 1: DEADLOCKS (The Traffic Jam)

Found in: Chapter 7 and almost every past exam (Q1 in many papers).

1. The Concept (Story Mode)

Imagine 4 cars at a 4-way intersection. Every car moves forward a bit, blocking the other. No one can move back. No one can move forward. This is a **Deadlock**.
In a computer, **Processes** (programs) fight for **Resources** (printers, memory, disk). If Process A holds the Printer and waits for the Disk, but Process B holds the Disk and waits for the Printer, they wait forever.

2. The 4 Necessary Conditions

For a deadlock to happen, you need **ALL 4** of these things to be true. If you break just one, you fix the deadlock. Memorize these:

1. **Mutual Exclusion:** Only one person can use the resource at a time (like a non-shareable Printer).
2. **Hold and Wait:** You hold a resource while waiting for a new one. (You are greedy).
3. **No Preemption:** No one can steal the resource from you. You must give it up voluntarily.

Circular Wait: A circle of waiting.
P1→P2→P3→P1P1 \to P2 \to P3 \to P1P1→P2→P3→P1

4.
.

3. Exam Task: The RAG (Resource Allocation Graph)

You will see a diagram with Circles and Squares.

Circle (
PPP

): A Process (e.g.,
P1,P2P_1, P_2P1,P2

-).

Square (
RRR

): A Resource type (e.g.,
R1,R2R_1, R_2R1,R2

-).

Dots inside Square: Instances (Copies). If
R1R_1R1

- has 3 dots, there are 3 copies of that resource.

How to Read the Arrows (Edges):

Request Edge (
$P \to RP \to R$

- **):** Process needs resource. Arrow goes OUT of circle.

Assignment Edge (
$R \to PR \to P$

- **):** Process holds resource. Arrow goes INTO circle.

The Golden Rule:

- If you see a **Cycle** (a closed loop of arrows), **Deadlock MIGHT exist**.
- If resources have only **1 dot (instance)**: Cycle = Deadlock (100%).
- If resources have **multiple dots**: Cycle = Deadlock is **Possible**, but not guaranteed (you must check if another process can finish and return a resource to break the circle).

4. Exam Task: The Banker's Algorithm (High Marks!)

This is the math problem. You must memorize this table logic.
You will be given **Allocation** (what they have), **Max** (what they need total), and **Available** (what the system has free).

The Magic Formula:

Need=Max−AllocationNeed = Max - AllocationNeed=Max−Allocation

How to Solve the "Safe Sequence" Question:

1. **Step 1:** Create the **Need** Matrix using the formula above. Subtract Allocation from Max for every row.
2. **Step 2 (The Game):** Look at your **Available** resources (your money in hand).

Step 3: Find a Process (
PPP

) whose **Need** is
≤\le≤

3.
Available.
   - *Logic:* Can I afford to give this guy what he wants?
4. **Step 4 (Simulate):** If Yes:
   - Pretend the process runs and finishes.
   - Take his **Allocation** and add it back to **Available**.
   - New Available = Old Available + Process Allocation.
   - Mark this Process as "Done".
5.
6. **Step 5:** Repeat until all processes are done.

Result: The order you finished them (e.g.,
<P1,P3,P0,P2,P4><P_1, P_3, P_0, P_2, P_4><P1,P3,P0,P2,P4>

7.
) is the **Safe Sequence**.

Request Algorithm Variant:

"Process
P1P_1P1

- requests (1, 0, 2). Can we grant it?"

Check 1: Is Request
≤\le≤

- Need? (Is he asking for more than he promised?)

Check 2: Is Request
≤\le≤

- Available? (Do we have it in stock?)
- **Action:** If both yes, temporarily **pretend** to give it:
    - Available = Available - Request
    - Allocation = Allocation + Request
    - Need = Need - Request
- **Test:** Now run the Safety Algorithm (Step 2 above) with these *new numbers*. If it works, say **"Yes, Granted."** If not, say **"No, Unsafe."**

MODULE 2: MEMORY ALLOCATION (Filling the Box)

Found in: Chapter 8 and questions involving partitions, holes, and fragmentation.

1. Fragmentation (The Wasted Space)

- **Internal Fragmentation:** The hole is inside the box.
    - *Example:* You put a 30KB job into a 50KB fixed box. There is 20KB wasted inside. This happens in **Fixed Partitioning**.
- **External Fragmentation:** The hole is outside (between) boxes.
    - *Example:* You have 50KB free space here, and 50KB free space there. Total 100KB free. A job needs 90KB. You can't fit it because the space is not together (contiguous). This happens in **Dynamic Partitioning**.

2. Allocation Strategies (Placing the Job)

The exam will give you a list of holes (empty spaces) and a list of jobs.

- **First-Fit:** Put the job in the **first** hole that is big enough.
    - *Pros:* Fast.
- **Best-Fit:** Put the job in the hole that is **closest** in size (leaves the smallest leftover).
    - *Pros:* Saves large holes for large jobs. *Cons:* Leaves tiny, useless scraps.
- **Worst-Fit:** Put the job in the **largest** hole available.
    - *Pros:* The leftover space is big enough to be useful. *Cons:* Wastes big spaces quickly.

-

Tip: Usually, First-Fit and Best-Fit are better than Worst-Fit.

3. Compaction

If we have many small holes (External Fragmentation), we shuffle all the files to one side of memory to make one big empty block. This is called **Compaction**.

- **Requirement:** Memory addresses must be dynamic (moveable) during runtime.

MODULE 3: PAGING (The Jigsaw Puzzle)

Found in: Chapter 8 & 9. Often calculation questions in Section B.

1. The Concept

Instead of finding a big contiguous space, we cut the program into small equal pieces called **Pages**. We cut the physical RAM into slots of the same size called **Frames**.
We can put any Page into any Frame.

- **Logical Address** = CPU view (Pages).
- **Physical Address** = RAM view (Frames).

2. Essential Formulas (Memorize these!)

Logical Address Size =
2m2^m2m

bytes. (Means address has
mmm

- bits).

Page Size =
2n2^n2n

bytes. (Means offset
ddd

has
nnn

- bits).

Page Number bits (
ppp

) =
m−nm - nm−n

- .
- **Total Entries in Page Table** = Total Pages.

Physical Address = (Frame Number
×\times×

- Frame Size) + Offset.

3. Converting Logical to Physical (The Math)

Exam Scenario: "Logical address is 300. Page size is 100. Page Table says Page 3 is in Frame 8."

Calculate Page Number (
`ppp`

):
300/100=3300 / 100 = 3300/100=3

1. .

Calculate Offset (
`ddd`

):
300(mod100)=0300 \pmod{100} = 0300(mod100)=0

2. .
3. **Look up Table:** Page 3 is in Frame 8.

Calculate Physical:
(8×100)+0=800(8 \times 100) + 0 = 800(8×100)+0=800

4. .

MODULE 4: VIRTUAL MEMORY & PAGE REPLACEMENT

Found in: Chapter 9. Almost always Q3 or Q4 in the exam.

1. Demand Paging

We don't load the whole program. We only load pages when we need them ("Demand" them).

- **Page Fault:** We try to access a page, but the valid-invalid bit is **i (invalid/not in RAM)**. The OS must stop, fetch it from the disk, and put it in a free frame.
- **Thrashing:** The computer is so busy swapping pages in and out (paging) that it does no real work. Caused by not having enough frames for the working set.
- **Preventing Thrashing:** Decrease the "degree of multiprogramming" (stop running so many programs at once).

2. Page Replacement Algorithms (MUST KNOW!)

The RAM is full. Who do we kick out? The exam will give you a **Reference String** (e.g., 7, 0, 1, 2, 0, 3...) and a number of Frames (e.g., 3).

A. FIFO (First-In, First-Out)

- The queue method.
- The page that came in first is the first to leave.
- **Belady's Anomaly:** A weird bug where giving FIFO *more* frames actually causes *more* faults.

B. OPT (Optimal)

- The Psychic method (Magic).
- Look **Forward** (to the right) in the string.
- Kick out the page that will **not be used for the longest time** in the future.
- *Note:* This is impossible in real life, but good for comparison. It gives the **minimum** faults.

C. LRU (Least Recently Used)

- The History method.
- Look **Backward** (to the left) in the string.
- Kick out the page that was **unused for the longest time** in the past.
- This is the best realistic algorithm.

How to calculate Page Faults in Exam:

1. Draw a grid. Rows = Frames, Columns = Reference String numbers.
2. Fill numbers.
3. If a number is already in the column, **Hit** (No fault). Do nothing (for FIFO) or update time (for LRU).
4. If a number is not there, **Fault**. Kick someone out based on the rule.
5. Count the faults at the end.

MODULE 5: FILE SYSTEM INTERFACE & PROTECTION

Found in: Chapter 10. Usually multiple-choice or short questions.

1. Access Control (Permissions)

UNIX uses 3 groups: **Owner**, **Group**, **Universe (Public/World)**.
Permissions are: **Read (r)**, **Write (w)**, **Execute (x)**.

The Math (Octal):

r=4r = 4r=4

-

w=2w = 2w=2

-

x=1x = 1x=1

-

–-–

(no access)
=0= 0=0

-

Exam Question: "What does chmod 751 mean?"

First digit (Owner): 7 = 4+2+1 (rwx)
→\to→

-
  Read, Write, Execute.

Second digit (Group): 5 = 4+0+1 (r-x)
→\to→

-
  Read, Execute.

Third digit (Public): 1 = 0+0+1 (--x)
→\to→

-
  Execute only.

2. Directory Structures

- **Single-Level:** Everyone in one big bucket. Naming collisions! (I can't name my file test if you already did).
- **Two-Level:** Each user has their own directory. Solves collisions, but hard to share.

Tree-Structured: The standard folder inside a folder (
/home/user/docs//home/user/docs//home/user/docs/

-). Good grouping.
- **Acyclic Graph:** Allows sharing files/folders (using links/aliases). Danger: Deleting a file might leave a "dangling pointer" if not careful.

MODULE 6: FILE IMPLEMENTATION & ALLOCATION

Found in: Chapter 11. Often diagrams or filling in tables.

1. Contiguous Allocation

- File occupies blocks 1, 2, 3... together.
- **Good:** Fast sequential and direct access.
- **Bad:** External fragmentation (holes). Hard to grow file size.

2. Linked Allocation

- Block 1 points to Block 5, Block 5 points to Block 2.
- **Good:** No external fragmentation. Easy to grow.
- **Bad:** Slow direct access (must follow the chain). Reliable (if a pointer breaks, file is lost).
- **FAT (File Allocation Table):** A variation used in Windows/USB drives. The pointers are stored in a separate table at the start of the disk, not in the blocks.

3. Indexed Allocation (Unix Inodes)

- A special **Index Block** contains a list of pointers to all other blocks.
- **Good:** Fast direct access. No external fragmentation.
- **Bad:** Wasted space for the index block.

MODULE 7: MASS STORAGE & DISK SCHEDULING

Found in: Chapter 12. Guaranteed math question in Part B.

1. The Disk Moving Problem

The disk arm (Head) acts like an elevator. It needs to pick up data (people) from different Cylinders (floors).
Goal: Minimize total head movement (Total Distance).

2. The Algorithms

The exam gives: Current Head Position (e.g., 53), Request Queue (e.g., 98, 183, 37...), and disk size (0-199).

A. FCFS (First-Come, First-Served):

- Go to 98, then 183, then 37... exactly in order.

Total Distance =
$|53-98|+|98-183|+\ldots$|53 - 98| + |98 - 183| + \dots$|53-98|+|98-183|+\ldots$

- (It's wild and slow).

B. SSTF (Shortest Seek Time First):

- Go to the **closest** number to where you are.

From 53, is 37 closer or 98 closer?
$|53-37|=16$|53-37|=16$|53-37|=16$

.
$|53-98|=45$|53-98|=45$|53-98|=45$

- . 37 is closer. Go to 37.
- Repeat from 37.
- *Warning:* Requests far away might never get served (**Starvation**).

C. SCAN (The Elevator):

- Go all the way to the end (0 or 199), picking up requests along the way. Then reverse.
- **Direction matters!** Exam will say "moving toward 0" or "moving toward 199".

Example (moving to 0): 53
$\rightarrow$\to$\rightarrow$

37
$\rightarrow$\to$\rightarrow$

14
→\to→

0 (Hit the wall!)
→\to→

reverse
→\to→

65
→\to→

- 67...

D. C-SCAN (Circular Elevator):

- Like SCAN, go to the end. But when you hit the end (199), **jump immediately to 0** without serving anything.
- Imagine a one-way loop.

53
→\to→

65
→\to→

...
→\to→

199
→\to→

0 (The Jump)
→\to→

14
→\to→

- 37.
- *Reason:* More uniform wait time.

E. C-LOOK (The Smart Circular Elevator):

- Like C-SCAN, but **don't go to the wall**.
- Go to the last request (e.g., 183), then jump to the first request (e.g., 14). Do not touch 199 or 0 unless a request is there.

3. RAID (Redundant Array of Independent Disks)

We combine multiple disks to be **Safe** or **Fast**.

- **RAID 0 (Striping):** Speed only. Data is split across disks. If one breaks, data is lost. **No Redundancy.**
- **RAID 1 (Mirroring):** Safety. Data is copied 100% to a second disk. Very expensive (need 2x storage). **High Redundancy.**
- **RAID 5 (Striping + Parity):** Best of both. Data is striped, and a "Parity" block (safety code) is spread across all disks. If one disk fails, math is used to recover the data.

"MAGIC PAPER" SUMMARY: How to pass tomorrow?

1. **Don't panic.** The concepts are physical things. Disks are elevators. Memory is boxes. Files are papers in folders.
2. **Practice the 4 Calculations:**
    - **Banker's:** Max - Alloc = Need. Check Available >= Need.

Paging: Logical size
///

- Page Size = number of pages. Table lookup.
- **Page Replacement:** Draw the frames. Count the faults. Remember LRU looks back, OPT looks forward.

Disk: Draw a number line. Plot the points. Calculate distance (
|A−B||A - B||A−B|

-). Check if it's SCAN (touch 0) or LOOK (touch last request).

3.
4. **Memorize UNIX Rights:** r=4, w=2, x=1.
5. **Know your definitions:**
    - **Deadlock:** Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait.
    - **Thrashing:** Too much paging, low CPU. Fix by suspending processes.
    - **Fragmentation:** Internal (in page) vs. External (dynamic hole).

MMU: Hardware translator (Virtual
→\to→

- Physical).

6.

Specific Trap to Avoid:
In C-SCAN disk scheduling, when the head jumps from the End to the Start (e.g., 199 to 0), **that distance counts**! Don't forget to add

(199−0)(199-0)(199−0)

or whatever the boundary is. However, C-LOOK jumps from Last Request to First Request, and that distance also counts.

You are now equipped with the core logic of the past exams. Good luck!